



# Hashing & Hash Tables

Foundations, Uniqueness, Frequencies & Core Applications

# Session Agenda

---

## Idea 1: Core Foundations of Hashing

The  $\mathcal{O}(1)$  Search Dream, String ASCII Mapping, Modulo Compression & Collision Resolution (Chaining vs Probing)

## Idea 2: The Uniqueness Mindset & HashSets

HashSet Architecture, the "Seen" Pattern, Cycle Detection & Sequence Anchors with Variations Summary

## Idea 3: Frequency Maps & Running Counters

Associative Counters, Delta Balancing, Two Sum Complement Search & Prefix Sum Subarrays with Visual Dry Runs

## Idea 4: Applications: Rabin-Karp & LRU Cache

Rolling Hash for  $\mathcal{O}(N)$  Substring Search and Combining Doubly Linked Lists with HashMaps for  $\mathcal{O}(1)$  Cache Systems

Idea 1

# Core Foundations of Hashing

From the Search Dilemma to ASCII Mapping, Modulo Compression & Collision Resolution

Chapter 03: Hashing | Core Mental Model Masterclass

# The Fundamental Search Dilemma

## The Search Challenge

Given a collection of  $N$  items, we constantly need:

- **Search( $k$ )**: Does key  $k$  exist?
- **Insert( $k$ )**: Add a new key  $k$ .
- **Delete( $k$ )**: Remove key  $k$ .

Can we achieve **instant**  $\mathcal{O}(1)$  **time** for all three?

## Standard Data Structures

- **Unsorted Array**:  $\mathcal{O}(1)$  insert, but search requires linear scan  $\rightarrow \mathcal{O}(N)$ .
- **Sorted Array**:  $\mathcal{O}(\log N)$  binary search, but inserting requires shifting  $\rightarrow \mathcal{O}(N)$ .
- **BST**:  $\mathcal{O}(\log N)$  search/insert, but pointer overhead and balancing required.

Invariant

Can we eliminate search loops entirely and jump straight to the data in  $\mathcal{O}(1)$  time?

# The Dream: What if Keys Were Array Indices?

## The Physical Array Superpower

In physical computer memory, an array gives **instant**  $\mathcal{O}(1)$  **random access**:

$$\text{Address}(\text{arr}[i]) = \text{Base} + i \times \text{Size}$$

If our keys are small numbers (e.g. 0, 1, 2, 3):

- Store key 2  $\implies$  write to `table[2]`.
- Search key 2  $\implies$  check `table[2]`.

Zero loops. Pure  $\mathcal{O}(1)$  lookup!

## The Real-World Dilemma

What if our keys are **NOT** small numbers?

- What if the key is a string: "cat", "apple", "username"?
- What if keys are phone numbers ( $10^{10}$ )?
- We cannot allocate an array of size  $10^{10}$ !

**Invariant**

How can we convert arbitrary strings and keys into valid array indices?

# First Intuition: Mapping Characters to ASCII Numbers

## Every Character is Already a Number

Computers store characters as ASCII/Unicode integers:

- 'c'  $\implies$  99
- 'a'  $\implies$  97
- 't'  $\implies$  116

**Baby Hash Idea:** Why not simply **sum the ASCII values** of all letters in the word?

## The Rudimentary Hash Function

For the word "cat":

$$\text{Hash}(\text{"cat"}) = 99 + 97 + 116 = 312$$

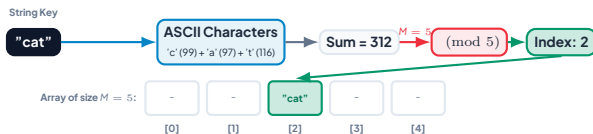
We converted an arbitrary string into an integer!

**Problem:** We still don't want an array of size 313 just for a 3-letter word!

**Invariant**

A hash function translates arbitrary objects into integer hash codes.

# Compressing to Array Size: The Modulo Function ( $\text{mod } M$ )



## Step 1: Compute Hash Code

Convert string "cat" into integer:  $99 + 97 + 116 = 312$ .

## Step 2: Modulo Compression

Compress into compact array of size  $M = 5$ :

$$\text{Index} = 312 \quad (\text{mod } 5) = 2$$

Store "cat" directly in slot 2!

**Invariant**

Modulo compression maps infinitely large numbers into a small, bounded array index  $[0 \dots M - 1]$ .

# The Inevitable Problem: Collisions!

## What Happens with "act"?

Now suppose we want to insert the word "act":

- Character sum:  
 $'a'(97) + 'c'(99) + 't'(116) = 312!$
- Modulo index:  $312 \pmod{5} = 2!$

Both "cat" and "act" desire the exact same slot `table[2]`!

## The Pigeonhole Principle

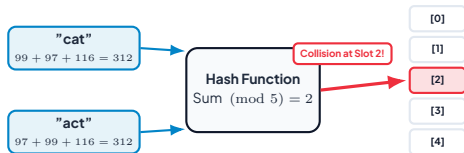
Because there are infinitely many strings and only  $M = 5$  array slots, collisions are **mathematically unavoidable**!

Collisions are not a bug — they are a mathematical certainty. How do we resolve them?

**Invariant**

**When two different keys produce the same array index, a collision occurs.**

# Visualizing the Collision



## Step Deduction – The Core Conflict

Both "cat" and "act" calculate index 2. If we blindly overwrite `table[2]`, "cat" is permanently erased! We need a resolution strategy.

# Collision Resolution 1: Separate Chaining

## The Intuition: Don't Fight!

Instead of letting elements overwrite each other:

- Each array slot is a **bucket** holding a little **linked list**.
- When "cat" arrives at slot 2 → add to list.
- When "act" arrives at slot 2 → append to list!

Both keys live peacefully in the same bucket!

## How Searching Works

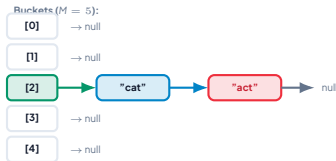
To search for "act":

1. Calculate index:  $312 \pmod{5} = 2$ .
2. Jump straight to bucket 2 in  $\mathcal{O}(1)$  time.
3. Walk the tiny list at bucket 2:
  - Compare "cat"  $\neq$  "act" → next.
  - Compare "act"  $==$  "act" → **Found!**

**Invariant**

Separate Chaining stores colliding elements in a linked list attached to each array slot.

# Separate Chaining: Step-by-Step Visual



## Step Deduction – Chain Invariant

Slot 2 holds a linked list: "cat" -> "act" -> null. When hash distributions are uniform, chain lengths are tiny (1–2 elements), keeping lookups essentially  $\mathcal{O}(1)$ .

# Collision Resolution 2: Linear Probing

## The Intuition: Find the Next Chair

What if we don't want linked lists or pointers?

- Everything stays in a single flat array!
- When "cat" is inserted → sits in slot 2.
- When "act" arrives → sees slot 2 occupied!
- **Rule:** Check the next seat: index 3.
- Slot 3 is empty → place "act" in slot 3!

## How Searching Works

To search for "act":

1. Calculate preferred slot:  $312 \pmod{5} = 2$ .
2. Check slot 2: holds "cat"  $\implies$  not a match!
3. Probe next slot 3: holds "act"  $\implies$  **Found!**
4. If an empty slot is encountered  $\rightarrow$  key does not exist!

Invariant

Linear Probing resolves collisions by sequentially scanning for the next empty array slot.

# Linear Probing: Step-by-Step Visual

Linear Probing Array ( $M = 5$ ): Slot 2 occupied  $\rightarrow$  hop to 3



"act" wanted slot 2, found it occupied by "cat", and moved to the next free chair at slot 3!

## Step Deduction – Probing Mechanics

No pointers or extra memory nodes are required. Elements stay contiguous in memory, which is exceptionally fast for CPU cache access!

# Chaining vs. Probing: At a Glance

## Separate Chaining (Lists)

- Array of linked lists / buckets.
- Elements never displace other buckets.
- Can store more items than table size ( $N > M$ ).
- Extra memory for node pointers.
- Used by Java HashMap & HashSet.

## Linear Probing (Flat Array)

- Pure flat array (zero pointers).
- Colliding items step into adjacent slots.
- Cannot exceed array capacity ( $N < M$ ).
- Outstanding CPU L1 cache locality.
- Used by Python dictionaries.

Invariant

Chaining offers robust bucket isolation; Probing offers contiguous cache speed.

## Idea 2

# The Uniqueness Mindset & HashSets

From Mathematical Set Invariants to  $\mathcal{O}(1)$  Membership, Deduplication & Sequence Anchors

# The Uniqueness Trigger in Algorithmic Problems

## Problem Language Triggers

Whenever an interview problem statement asks for:

- "Find distinct / unique elements..."
- "Check if an array contains duplicates..."
- "Has this exact state been visited before?..."
- "Find the intersection of two collections..."
- "Detect whether a sequence loops / cycles..."

Your immediate intuition: **Set Invariants!**

## The Two Set Guarantees

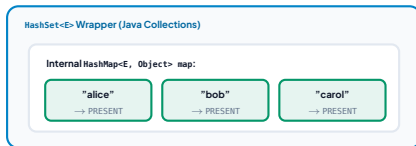
A mathematical Set provides two ironclad properties:

1. **Strict Uniqueness:** Elements are inherently distinct. Duplicates cannot exist.
2.  $\mathcal{O}(1)$  **Membership Oracle:** Querying whether  $x \in S$  takes  $\mathcal{O}(1)$  average time.

**Invariant**

A HashSet is an  $\mathcal{O}(1)$  query oracle that answers: "Have I seen this element before?"

# Under the Hood: How Java's HashSet is Built



## A Wrapper Over HashMap

In `java.util.HashSet<E>`, elements are NOT stored in a custom set structure:

- Each element `e` is stored as a **Key** in an internal `HashMap<E, Object>`.
- The map value is a dummy static sentinel `PRESENT`.

## Why Map Keys Enforce Uniqueness

Because `HashMap`s forbid duplicate keys, `HashSet` inherits collision resolution, separate chaining, and  $\mathcal{O}(1)$  lookup for free!

**Invariant**

**HashSet reuses the collision resolution and  $\mathcal{O}(1)$  indexing of HashMap.**

# Java HashSet API & The .add() Invariant

```
1 Set<Integer> set = new HashSet<>();
2
3 // 1. Adding: returns true if new, false if dup
4 boolean first = set.add(42); // true
5 boolean second = set.add(42); // false (DUP!)
6
7 // 2. Checking membership in O(1) avg
8 boolean exists = set.contains(42); // true
9
10 // 3. Deleting in O(1) avg
11 boolean removed = set.remove(42); // true
```

## The Powerful .add() Idiom

Notice that `set.add(x)` performs **both** existence checking AND insertion in one step:

```
if (!set.add(x))
```

Executes an  $O(1)$  duplicate check and immediately signals if `x` was seen!

ONE-STEP DUPLICATE DETECTION

# The Canonical Pattern: The "Seen" Set Filtering Template

```
1 public List<Integer> deduplicate(int[] nums) {  
2     Set<Integer> seen = new HashSet<>();  
3     List<Integer> result = new ArrayList<>();  
4     for (int x : nums) {  
5         if (seen.add(x)) { // true on first sight  
6             result.add(x);  
7         }  
8     }  
9     return result;  
10 }
```

## The "Seen" Set Invariant

- **State:** seen contains all distinct values processed so far.
- **Transition:** For element  $x$ , if  $x \notin \text{seen}$ , record  $x$  and accept it.
- **Complexity:**  $\mathcal{O}(N)$  total time instead of  $\mathcal{O}(N^2)$  nested scans!

Invariant

Maintaining a "seen" set converts  $\mathcal{O}(N^2)$  brute-force comparisons into a single  $\mathcal{O}(N)$  pass.

# Stream Deduplication: Visual Execution

```
1 // Input: [5, 2, 8, 2, 5, 1]
2 for (int x : nums) {
3     // seen.add(2) on idx 3 -> returns false!
4     if (seen.add(x)) result.add(x);
5 }
6 // Result: [5, 2, 8, 1]
```

Stream: 5, 2, 8, [2(DUP)], [5(DUP)], 1  
| Set={1,2,5,8}

InputStream: [5, 2, 8, 2, 5, 1]



Final HashSet: {1, 2, 5, 8} (Guaranteed Unique)

## Step Deduction – Deduplication Mechanics

When encountering the second '2' at index 3, 'seen.add(2)' returns 'false' in  $\mathcal{O}(1)$  time. The duplicate is rejected without any array scanning!

# Variation 1: Duplicate Detection & Early Exit (LC 217)

```
1 public boolean containsDuplicate(int[] nums) {  
2     Set<Integer> seen = new HashSet<>();  
3     for (int num : nums) {  
4         if (!seen.add(num)) {  
5             return true; // Early exit on duplicate!  
6         }  
7     }  
8     return false; // All elements unique  
9 }
```

## The Early-Exit Optimization

- **Sorting Approach:**  $\mathcal{O}(N \log N)$  time, modifies array.
- **HashSet Approach:**  $\mathcal{O}(N)$  worst-case, but  $\mathcal{O}(1)$  **best-case** if duplicate occurs early!
- Demonstrates the power of immediate failure triggers.

**Invariant**

The "seen" set allows instant early return without scanning remaining inputs.

# Variation 2: Cycle Detection in State Graphs (LC 202)

```
1 public boolean isHappy(int n) {
2     Set<Integer> seen = new HashSet<>();
3     while (n != 1 && seen.add(n)) {
4         n = getSumOfSquares(n);
5     }
6     return n == 1; // 1 -> Happy, cycle -> Unhappy
7 }
8 private int getSumOfSquares(int n) {
9     int sum = 0;
10    while (n > 0) {
11        int d = n % 10;
12        sum += d * d;
13        n /= 10;
14    }
15    return sum;
16 }
```

## State Cycle Detection

**Key Insight:** In dynamic processes, a cycle occurs whenever the current state equals a previously seen state.

`seen.add(n)` automatically halts the simulation as soon as a duplicate state is detected!

Invariant

Cycle detection in discrete math simulations is fundamentally a "seen set" problem.

# Variation 3: Set Intersection & Membership Queries (LC 349)

```
1 public int[] intersection(int[] nums1, int[] nums2) {
2     Set<Integer> set1 = new HashSet<>();
3     for (int x : nums1) set1.add(x);
4
5     Set<Integer> resultSet = new HashSet<>();
6     for (int x : nums2) {
7         if (set1.contains(x)) resultSet.add(x);
8     }
9     int[] res = new int[resultSet.size()];
10    int i = 0; for (int x : resultSet) res[i++] = x;
11    return res;
12 }
```

## Dual-Set Filtering

- **Step 1:** Convert `nums1` into an  $\mathcal{O}(1)$  query oracle `set1`.
- **Step 2:** Scan `nums2`, querying `set1.contains(x)`.
- **Guaranteed Uniqueness:** Adding matches to `resultSet` eliminates duplicates.

Invariant

Pre-populating a set turns arbitrary containment checks into  $\mathcal{O}(1)$  operations.

# Variation 4: The Sequence Anchor Pattern (LC 128)

## The Challenge (LC 128)

Given an unsorted array of integers:

[100, 4, 200, 1, 3, 2]

Find the length of the **longest consecutive elements sequence**.

Consecutive streak: [1, 2, 3, 4] → **Length = 4**.

**Constraint:** Must run in  $\mathcal{O}(N)$  time!

## Why Sorting Fails

- Sorting array takes  $\mathcal{O}(N \log N)$  time  $\implies$  fails strict  $\mathcal{O}(N)$  constraint!
- If we put all elements in a HashSet, how can we find streaks without scanning every number's neighbors?

**Invariant**

Can we find consecutive chains in unsorted data without sorting?

# The Anchor Invariant: Identifying Streak Starts

HashSet: {100, 4, 200, 1, 3, 2} → Anchor Check: !set.contains(x - 1)



## The Streak Anchor Condition

A number  $x$  is the **beginning (anchor)** of a streak IF AND ONLY IF:

$$(x - 1) \notin \text{Set}$$

If  $(x - 1) \in \text{Set}$ ,  $x$  is part of a larger streak → **SKIP**  $x$ !

## Why This Guarantees $\mathcal{O}(N)$

We only expand forward from true anchors. Non-anchor elements are skipped immediately. Every number is visited at most twice!

Invariant

Checking ' $!set.contains(x - 1)$ ' prunes redundant inner loop expansions entirely.

# Longest Consecutive Sequence: Java Implementation

```
1 public int longestConsecutive(int[] nums) {
2     Set<Integer> set = new HashSet<>();
3     for (int x : nums) set.add(x);
4
5     int maxStreak = 0;
6     for (int x : set) {
7         // Only start expanding if 'x' is anchor!
8         if (!set.contains(x - 1)) {
9             int currNum = x, currStreak = 1;
10            while (set.contains(currNum + 1)) {
11                currNum++;
12                currStreak++;
13            }
14            maxStreak = Math.max(maxStreak, currStreak);
15        }
16    }
17    return maxStreak;
18 }
```

## Execution Invariant

- For {100, 200, 1}, outer loop checks 99, 199, 0  $\notin$  S.
- Only 1 triggers the inner while loop, hopping 1  $\rightarrow$  2  $\rightarrow$  3  $\rightarrow$  4.
- Numbers 2, 3, 4 are skipped in  $\mathcal{O}(1)$  time when iterated over.

STRICT  $\mathcal{O}(N)$  RUNTIME

# Variation 5: Fixed Universe & Pangram Checks (LC 1832)

```
1 // HashSet approach: General universe
2 public boolean checkIfPangram(String s) {
3     Set<Character> seen = new HashSet<>();
4     for (char c : s.toCharArray()) seen.add(c);
5     return seen.size() == 26;
6 }
7 // Array optimization: Bounded universe (a-z)
8 public boolean checkIfPangramArray(String s) {
9     boolean[] seen = new boolean[26];
10    int count = 0;
11    for (char c : s.toCharArray()) {
12        if (!seen[c - 'a']) { seen[c - 'a'] = true; count++; }
13    }
14    return count == 26;
15 }
```

## Set vs. Boolean Array

- **General Keys:** Use HashSet for unbounded/composite types (Strings, Objects).
- **Bounded Universe:** When keys are strictly lowercase English letters (a..z), a boolean[26] array eliminates heap allocation overhead.

Invariant

When the universe is small and contiguous, fixed boolean arrays provide zero-allocation set behavior.

# Idea 2 Variations: Uniqueness Mental Model Table

| Problem & LeetCode                  | Uniqueness Trigger       | HashSet Invariant Role                           | Time               | Space              |
|-------------------------------------|--------------------------|--------------------------------------------------|--------------------|--------------------|
| <b>Contains Duplicate</b> (LC 217)  | Check duplicate          | Early-exit if <code>!seen.add(x)</code>          | $\mathcal{O}(N)$   | $\mathcal{O}(N)$   |
| <b>Happy Number</b> (LC 202)        | State cycle detection    | Terminate simulation if state is re-added        | $\mathcal{O}(K)$   | $\mathcal{O}(K)$   |
| <b>Array Intersection</b> (LC 349)  | Shared distinct items    | Convert array 1 to $\mathcal{O}(1)$ query oracle | $\mathcal{O}(N+M)$ | $\mathcal{O}(N+M)$ |
| <b>Longest Consecutive</b> (LC 128) | Unsorted sequence streak | Anchor check <code>!set.contains(x - 1)</code>   | $\mathcal{O}(N)$   | $\mathcal{O}(N)$   |
| <b>Pangram Check</b> (LC 1832)      | Complete alphabet seen   | Set size check vs <code>boolean[26]</code> array | $\mathcal{O}(N)$   | $\mathcal{O}(1)$   |

**Invariant**

All uniqueness problems boil down to querying whether an element has entered the seen state.

## Idea 3

# Frequency Maps & Running Counters

From Counting Invariants to Complement Lookups, Canonical Signatures & Prefix Sum Subarrays

# The Frequency Trigger in Algorithmic Problems

## Problem Language Triggers

Whenever an interview problem asks for:

- "Count occurrences / frequencies of items..."
- "Find elements appearing more than  $K$  times..."
- "Check if two strings are anagrams..."
- "Find two elements summing to target  $T$ ..."
- "Count contiguous subarrays with sum  $K$ ..."

Your immediate intuition: **Associative Counters**  
(HashMaps)!

## Set (Existence) vs. Map (State)

- **HashSet**: Stores **Keys only**. Answers boolean query: "Have I seen  $x$ ?"
- **HashMap**: Stores **Key  $\mapsto$  Value pairs**. Answers stateful queries:
  - "How many times has  $x$  appeared?" (Count)
  - "Where did  $x$  appear?" (Index)
  - "What group does  $x$  belong to?" (Bucket)

**Invariant**

A HashMap extends set existence into rich state tracking: counts, positions, and groups.

# Java HashMap Counter API & Modern Idioms

```
1 Map<Character, Integer> counts = new HashMap<>();
2
3 // 1. Classic Idiomatic Counter
4 counts.put(c, counts.getOrDefault(c, 0) + 1);
5
6 // 2. Functional Counter (Java 8+)
7 counts.merge(c, 1, Integer::sum);
8
9 // 3. Multi-value Grouping (Map<Key, List<Item>>)
10 map.computeIfAbsent(key, k -> new ArrayList<>()).add(item);
11
12 // 4. Safe read
13 int freq = counts.getOrDefault('a', 0);
```

## The getOrDefault() Idiom

Instead of verbose checking:  
if (!map.containsKey(k)) map.put(k, 0);  
map.put(k, map.get(k) + 1);  
getOrDefault(k, 0) + 1 handles absent  
keys and increments in a single clean  
expression!

IDIOMATIC COUNTER PATTERN

# Variation 1: First Unique Character (LC 387) — Motivation

## Problem Statement

Given a string `s = "loveleetcode"`, find the index of the **first non-repeating character**.

If no unique character exists, return -1.

- 'l' repeats later → not unique.
- 'o' repeats later → not unique.
- 'v' appears only once ⇒ **Answer = Index 2!**

## The Two-Pass Motivation

Why can't we solve this in one pass?

- When looking at index 0 ('l'), we don't know if another 'l' exists ahead!
- **Pass 1:** Build the complete frequency table across the whole string.
- **Pass 2:** Scan the string chronologically and pick the first char with count == 1!

Invariant

Pass 1 gathers global frequency counts; Pass 2 inspects order of appearance.

# Variation 1: First Unique Character — Java Code

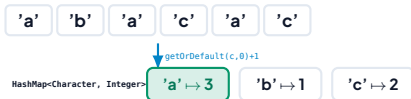
```
1 public int firstUniqChar(String s) {
2     Map<Character, Integer> count = new HashMap<>();
3
4     // Pass 1: Build frequency map
5     for (char c : s.toCharArray()) {
6         count.put(c, count.getOrDefault(c, 0) + 1);
7     }
8
9     // Pass 2: Find first character with frequency 1
10    for (int i = 0; i < s.length(); i++) {
11        if (count.get(s.charAt(i)) == 1) {
12            return i;
13        }
14    }
15    return -1;
16 }
```

## Complexity

- **Time:**  $2N$  steps  $\Rightarrow \mathcal{O}(N)$  total.
- **Space:**  $\leq 26$  entries  $\Rightarrow \mathcal{O}(1)$  space!

# Variation 1: Step-by-Step Visual Dry Run

Input String Stream: "abacac"



## Pass 1 Output (Frequencies):

'l': 1   'o': 2   'v': 1   'e': 4   't': 1   'c': 1   'd': 1

## Pass 2 Chronological Scan

Scan string: "loveleetcode"

1.  $i = 0, c = 'l' : \text{count}['l'] = 1 \implies \text{Return Index 0!}$

If string was "leetcode":

1.  $i = 0, 'l' : \text{count} = 1 \implies \text{return 0.}$
2. If duplicate at start, scan proceeds until first 1.

**Invariant**

Separating frequency aggregation from order scanning maintains pure  $\mathcal{O}(N)$  simplicity.

# Variation 2: Valid Anagram (LC 242) — Motivation

## Problem Statement

Given two strings  $s$  and  $t$ , return `true` if  $t$  is an **anagram** of  $s$ , else `false`.

Example:  $s = \text{"anagram"}, t = \text{"nagaram"}$ .

- Both contain: 3 'a's, 1 'n', 1 'g', 1 'r', 1 'm'.
- Exactly identical multiset of characters!

## The Delta Balance Motivation

Instead of building two maps and comparing them:

- Maintain a **single running balance map**.
- For every char in  $s \implies$  increment count by  $+1$ .
- For every char in  $t \implies$  decrement count by  $-1$ .
- If count drops below 0, characters diverge immediately!

**Invariant**

**Anagram checking is integer cancellation:  $+1$  on supply,  $-1$  on demand.**

# Variation 2: Valid Anagram — Java Code

```
1 public boolean isAnagram(String s, String t) {
2     if (s.length() != t.length()) return false;
3     Map<Character, Integer> map = new HashMap<>();
4
5     for (char c : s.toCharArray())
6         map.put(c, map.getOrDefault(c, 0) + 1);
7
8     for (char c : t.toCharArray()) {
9         int count = map.getOrDefault(c, 0);
10        if (count == 0) return false; // Insufficient supply!
11        map.put(c, count - 1);
12    }
13    return true; // Exactly balanced
14 }
```

## The Zero-Balance Invariant

Because lengths are checked equal up front, if no character ever drops below zero, every key must finish at exactly zero!

# Variation 2: Anagram Delta Balance Dry Run

## Pass 1: Supply Phase ( $s = \text{"rat"}$ )

- Character 'r'  $\Rightarrow$  'r': 1
- Character 'a'  $\Rightarrow$  'a': 1
- Character 't'  $\Rightarrow$  't': 1

Map: {'r': 1, 'a': 1, 't': 1}

## Pass 2: Demand Phase ( $t = \text{"car"}$ )

- First char 'c': Look up 'c'.
- `map.getOrDefault('c', 0) == 0!`
- **Mismatch:** 'c' was never supplied!
- **Early Exit:** Return false immediately!

Invariant

Early detection of deficit prevents scanning the rest of the second string.

# Variation 3: Two Sum (LC 1) — Motivation

## Problem Statement

Given array `nums = [2, 11, 7, 15]` and `target = 9`:  
Find two distinct indices  $i, j$  such that:

$$\text{nums}[i] + \text{nums}[j] == 9$$

**Brute Force:** Nested loops test all pairs  $\implies \mathcal{O}(N^2)$   
time!

## The Algebraic Inversion Motivation

Rearrange the formula:

$$\text{Complement} = \text{Target} - \text{nums}[i]$$

Instead of searching for a second number:

- At each element  $x$ , calculate what partner it needs:  
 $T - x$ .
- Check if that partner is already stored in our Map!

**Invariant**

Algebraic inversion turns a pairwise search into an  $\mathcal{O}(1)$  complement lookup.

# Variation 3: Two Sum — Java Code

```
1 public int[] twoSum(int[] nums, int target) {  
2     Map<Integer, Integer> map = new HashMap<>(); // num -> index  
3     for (int i = 0; i < nums.length; i++) {  
4         int complement = target - nums[i];  
5         if (map.containsKey(complement)) {  
6             return new int[] { map.get(complement), i };  
7         }  
8         map.put(nums[i], i);  
9     }  
10    throw new IllegalArgumentException("No pair");  
11 }
```

## The One-Pass Invariant

- **No Self-Pairing:** By querying complement **before** inserting `nums[i]`, an element can never match with itself.
- **Single Pass:** Matches are found as soon as the second partner appears!

# Variation 3: Two Sum Visual Dry Run

Target  $T = 9$  | Array: [2, 11, 7, 15]

At index  $i = 2$ , value  $x = 7$ :      Complement  $= 9 - 7 = 2$

$O(1)$  Match! ▼

Running Map {num  $\rightarrow$  index}:

2  $\mapsto$  index 0

11  $\mapsto$  index 1

Solution Found: Return indices [0, 2] in a single pass!

**Step 0** ( $i = 0, x = 2$ ):

Complement  $= 9 - 2 = 7$ .

7  $\notin$  map. Store {2: 0}.

**Step 1** ( $i = 1, x = 11$ ):

Complement  $= 9 - 11 = -2$ .

-2  $\notin$  map. Store {11: 1}.

**Step 2** ( $i = 2, x = 7$ ):

Complement  $= 9 - 7 = 2$ .

2  $\in$  map at index 0! Return [0, 2].

# Variation 4: Group Anagrams (LC 49) — Motivation

## Problem Statement

Given strings:

["eat", "tea", "tan", "ate", "nat", "bat"]

Group all anagrams together into sublists.

- "eat", "tea", "ate" → group 1.
- "tan", "nat" → group 2.
- "bat" → group 3.

## The Canonical Key Motivation

How do we know if two words belong in the same bucket?

- Map all permutations of a word to a **single canonical signature key**!
- If we sort the characters of each word:

$\text{sort}(\text{"eat"}) = \text{sort}(\text{"tea"}) = \text{"aet"}$

- All anagrams share the exact same sorted key!

**Invariant**

A canonical signature key routes heterogeneous words into the same hash bucket.

# Variation 4: Group Anagrams — Java Code

```
1 public List<List<String>> groupAnagrams(String[] strs) {  
2     Map<String, List<String>> map = new HashMap<>();  
3     for (String s : strs) {  
4         char[] chars = s.toCharArray();  
5         Arrays.sort(chars);  
6         String key = String.valueOf(chars); // Signature  
7  
8         map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);  
9     }  
10    return new ArrayList<>(map.values());  
11 }
```

## The Multi-Map Invariant

- **Key:** String (Canonical signature "aet").
- **Value:** List<String> (Bucket of words).
- `computeIfAbsent` initializes the list automatically on the first word!

# Variation 4: Canonical Key Grouping Dry Run

## Word-to-Signature Transformation

- "eat" → sort → "aet"
- "tea" → sort → "aet"
- "tan" → sort → "ant"
- "ate" → sort → "aet"
- "nat" → sort → "ant"
- "bat" → sort → "abt"

## Resulting Multi-Map Buckets

Bucket "aet":

→ ["eat", "tea", "ate"]

Bucket "ant":

→ ["tan", "nat"]

Bucket "abt":

→ ["bat"]

**Invariant**

Each distinct signature key naturally isolates a complete equivalence group.

# Variation 5: Subarray Sum Equals $K$ (LC 560) — Motivation

## Problem Statement

Given array `nums = [1, 2, 3, -2, 2]` and integer  $K = 3$ :  
Count the total number of contiguous subarrays whose sum equals  $K$ .

- Array can contain **negative numbers!**
- **Why Sliding Window Fails:** Negatives break window monotonicity (expanding can decrease the sum).

## The Prefix Sum Algebraic Equation

Subarray sum from index  $i + 1$  to  $j$ :

$$\text{Sum}(i + 1 \dots j) = P_j - P_i$$

We require this sum to equal  $K$ :

$$P_j - P_i = K \iff P_i = P_j - K$$

At current prefix  $P_j$ , how many previous prefix sums equal  $P_j - K$ ? A HashMap counter answers in  $\mathcal{O}(1)$ !

**Invariant**

Tracking running prefix sum frequencies solves arbitrary-sum subarray counts in linear time.

## Variation 5: Subarray Sum Equals $K$ — Java Code

```
1 public int subarraySum(int[] nums, int k) {
2     Map<Integer, Integer> map = new HashMap<>();
3     map.put(0, 1); // Base case: prefix 0 occurs once
4     int currSum = 0, count = 0;
5
6     for (int x : nums) {
7         currSum += x;
8         // Count previous prefixes satisfying  $P_i = P_j - K$ 
9         count += map.getOrDefault(currSum - k, 0);
10        // Record current prefix in running map
11        map.put(currSum, map.getOrDefault(currSum, 0) + 1);
12    }
13    return count;
14 }
```

**Caution – The Base Case:**  
**map.put(0, 1)**

Why initialize {0: 1}?

If a subarray starting at index 0 sums to  $K$ , its prefix sum is  $P_j = K$ .

Query:  $P_j - K = 0$ . Without {0 : 1}, prefix-rooted subarrays are missed!

# Variation 5: Prefix Sum Running Dry Run

Subarray Sum  $K = 3 \iff$  Equation:  $P_j - P_i = K \implies P_i = P_j - K$



Adding `map.get(3) = 2` discovers 2 valid subarrays ending at index  $j$ !

## Prefix Evolution on [1, 2, 3]

- Start: map = {0: 1}
- $x = 1, P = 1$ :  
 $1 - 3 = -2 \notin \text{map} \implies \text{map} = \{0 : 1, 1 : 1\}$
- $x = 2, P = 3$ :  $3 - 3 = 0 \in \text{map} \implies \text{count} += 1, \text{add } 3$
- $x = 3, P = 6$ :  $6 - 3 = 3 \in \text{map} \implies \text{count} += 1, \text{add } 6$

## Identified Subarrays

- Match at  $P = 3$ : Subarray [1, 2] (sum = 3).
- Match at  $P = 6$ : Subarray [3] (sum = 3).

Total Subarrays = 2 in single linear pass!

# Performance Deep Dive: HashMap vs. int[] Array

## When to Use HashMap<K, V>

- **Unbounded Keys:** Arbitrary integers, Large numbers ( $10^9$ ), Strings, Custom Objects.
- **Sparse Data:** Few elements scattered across massive range.
- **Dynamic Groups:** Nested collections (List<String>).

## When to Use int[] Counter

- **Bounded Alphabet:** ASCII ([0..127]) or Lowercase letters ([0..25]).
- **Zero Boxing Overhead:** No Integer object wrapper allocations on Java heap.
- **10x Faster:** Direct array indexing loads entirely into CPU L1 cache.

Invariant

For small fixed alphabets (a..z), 'int[26]' outperforms 'HashMap' by an order of magnitude.

# Idea 3 Variations: Frequency Mental Model Table

| Problem & LeetCode         | Frequency Trigger        | HashMap Invariant Role                                  | Time                     | Space             |
|----------------------------|--------------------------|---------------------------------------------------------|--------------------------|-------------------|
| First Unique Char (LC 387) | Distinct character count | Two-pass frequency tally                                | $\mathcal{O}(N)$         | $\mathcal{O}(1)$  |
| Valid Anagram (LC 242)     | Permutation equivalence  | Delta balance (+1 on s, -1 on t)                        | $\mathcal{O}(N)$         | $\mathcal{O}(1)$  |
| Two Sum (LC 1)             | Pair target sum          | Running map complement lookup ( $T - x$ )               | $\mathcal{O}(N)$         | $\mathcal{O}(N)$  |
| Group Anagrams (LC 49)     | Multi-item grouping      | Canonical signature key $\rightarrow$ list bucket       | $\mathcal{O}(NK \log K)$ | $\mathcal{O}(NK)$ |
| Subarray Sum = K (LC 560)  | Continuous sum count     | Running prefix map ( $P_i = P_j - K$ ) with $\{0 : 1\}$ | $\mathcal{O}(N)$         | $\mathcal{O}(N)$  |

Invariant

HashMaps transform quadratic pair, group, and subarray searches into single-pass  $\mathcal{O}(N)$  algorithms.

## Idea 4

# Applications: Rabin-Karp & LRU Cache

Rolling Hashes for  $\mathcal{O}(N)$  String Search & Composite Hash Systems

# String Matching: The Naive $\mathcal{O}(N \times M)$ Bottleneck

## The Substring Search Problem

Given text  $T$  of length  $N$  and pattern  $P$  of length  $M$  ( $M \leq N$ ):

- Find the first index where  $P$  appears in  $T$ .
- **Example:**
  - $T = \text{"acbaed"} (N = 6)$
  - $P = \text{"bae"} (M = 3)$
  - Expected Output: Index 2.

## Why the Naive Approach is Slow

- Slide pattern over every window  $i = 0 \dots N - M$ .
- For each window, compare characters one by one.
- **Worst-Case Scenario:**
  - $T = \text{"aaaa...a"} (N \text{ chars})$
  - $P = \text{"aaa...b"} (M \text{ chars})$
  - We check  $M$  characters before failing on every single window!
- Total Time:  $\mathcal{O}((N - M + 1) \times M) \approx \mathcal{O}(N \times M)$ .

Invariant

**Goal:** Can we compare an entire  $M$ -character window in  $\mathcal{O}(1)$  time instead of  $\mathcal{O}(M)$ ?

# The Mental Model: The Rolling Hash Odometer

## The Decimal Odometer Intuition

Imagine finding 3-digit number **325** in sequence "4325":

- Window 1: **4 3 2**  $\Rightarrow$  432.
- To slide to **3 2 5**, do we re-read all digits? **No!**
- **1. Drop Left:**  $432 - 4 \times 10^2 = 32$ .
- **2. Shift Left:**  $32 \times 10 = 320$ .
- **3. Add Right:**  $320 + 5 = 325$ !

Takes **3 arithmetic steps** =  $\mathcal{O}(1)$ !

## Odometer Transition Visual

Window 1: 4 3 2  $\Rightarrow$  432

$\mathcal{O}(1)$  Slide!

Window 2: 3 2 5  $\Rightarrow$  325

1. Drop Outgoing:  $432 - 4 \times 10^2 = 32$

2. Shift Left:  $32 \times 10 = 320$

3. Add Incoming (5):  $320 + 5 = 325$

Recomputing takes  $\mathcal{O}(1)$  time!

Invariant

A rolling hash slides across a window in  $\mathcal{O}(1)$  time by dropping the old char and adding the new.

# Polynomial Rolling Hash & Modulo Arithmetic

## Polynomial Hash Formula

Treat characters as digits in base  $B$  ( $B = 31$ ):

- **Window Hash:**

$$H = (\sum_{k=0}^{M-1} S[k] \cdot B^{M-1-k}) \pmod{Q}$$

- **Rolling Update Invariant:**

$$H_{\text{next}} = ((H_{\text{prev}} - S[\text{out}] \cdot B^{M-1}) \cdot B + S[\text{in}]) \pmod{Q}$$

- **Safe Modulo in Java:**

$$h = (h \% Q + Q) \% Q$$

## Spurious Hits & Verification

- **Why modulo  $Q$ ?**  $B^M$  overflows 64-bit integers. We use prime  $Q = 10^9 + 7$ .
- **Spurious Hit (Collision):** Two different strings can yield the same hash  $\pmod{Q}$ .
- **Two-Tier Check:**
  1.  $H_{\text{win}} \neq H_{\text{pat}} \implies$  **Guaranteed Mismatch** (skip!).
  2.  $H_{\text{win}} == H_{\text{pat}} \implies$  Verify actual characters in  $\mathcal{O}(M)$ .
- **Expected Runtime:**  $\mathcal{O}(N + M)$ !

Invariant

Modulo arithmetic prevents overflow; character verification guarantees 100% correctness.

# Rabin-Karp Implementation in Java

```
1 public class RabinKarp {
2     public static int search(String text, String pat) {
3         int n = text.length(), m = pat.length();
4         if (m > n) return -1;
5         long B = 31, Q = 1_000_000_007L, highPower = 1;
6         long patHash = 0, winHash = 0;
7
8         for (int i = 0; i < m - 1; i++) highPower = (highPower * B) % Q;
9         for (int i = 0; i < m; i++) {
10             patHash = (patHash * B + pat.charAt(i)) % Q;
11             winHash = (winHash * B + text.charAt(i)) % Q;
12         }
13         for (int i = 0; i <= n - m; i++) {
14             if (winHash == patHash && text.startsWith(pat, i)) return i;
15             if (i < n - m) {
16                 long drop = (text.charAt(i) * highPower) % Q;
17                 winHash = ((winHash - drop + Q) * B + text.charAt(i + m)) % Q;
18             }
19         }
20         return -1;
21     }
22 }
23 }
```

## Mechanics Summary

- **Precompute  $B^{M-1}$** : Drops outgoing character in  $\mathcal{O}(1)$ .
- **+ Q Safety**: Prevents negative modulo in Java.
- **startsWith(pat, i)**: Runs in  $\mathcal{O}(M)$  only when hashes match.

## Complexity:

- **Average Time**:  $\mathcal{O}(N + M)$
- **Worst Time**:  $\mathcal{O}(N \times M)$  (rare)
- **Space**:  $\mathcal{O}(1)$  auxiliary

Invariant

Rabin-Karp turns quadratic substring search into linear average time using an  $\mathcal{O}(1)$  rolling hash.

# Rabin-Karp: Step-by-Step Visual Dry Run

## Sliding Window Trace

Pattern  $P = \text{"bae"} (M = 3) \implies \text{Hash}(P) = 240$

Text  $T = \text{"acbaed"} (N = 6)$ :

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| a   | c   | b   | a   | e   | d   |
| [0] | [1] | [2] | [3] | [4] | [5] |

Win 0 [0..2]: "acb"

Hash = 105  $\neq$  240  $\implies$  Mismatch

Win 1 [1..3]: "cba"

Hash = 198  $\neq$  240  $\implies$  Mismatch

Win 2 [2..4]: "bae"

Hash = 240 == 240  $\implies$  MATCH at index 2!

## Execution Walkthrough

Target Pattern:  $P = \text{"bae"}, M = 3, \text{Hash}(P) = 240$ .

Text:  $T = \text{"acbaed"}, N = 6$ .

- **Window 0 [0..2] ("acb"):**  
Hash = 105  $\neq$  240  $\implies$  Skip immediately.
- **Window 1 [1..3] ("cba"):**  
Drop 'a', shift, add 'a'.  
Hash = 198  $\neq$  240  $\implies$  Skip immediately.
- **Window 2 [2..4] ("bae"):**  
Drop 'c', shift, add 'e'.  
Hash = 240 == 240  $\implies$  Hash Match!  
Verify characters: "bae" == "bae"  $\implies$  **Return Index 2!**

**Invariant**

Mismatches are discarded in  $\mathcal{O}(1)$  without comparing strings; exact check triggers only on hash match.

# LRU Cache: The $\mathcal{O}(1)$ Design Dilemma

## The LRU Cache Specification

Design a Least Recently Used (LRU) Cache:

- `get(key)`: Return value if present, else `-1`. Promotes key to **Most Recently Used**.
- `put(key, value)`: Insert or update key-value.
- **Capacity Invariant**: If size exceeds limit, **evict the Least Recently Used item**.
- **Goal**: Both operations MUST run in **strictly  $\mathcal{O}(1)$  time!**

## Why Single Data Structures Fail

- **LinkedList alone**: Preserves recency order and enables  $\mathcal{O}(1)$  head insertion / tail deletion, **but finding a key requires an  $\mathcal{O}(N)$  scan!**
- **HashMap alone**: Provides instant  $\mathcal{O}(1)$  key lookup, **but has zero chronological order**; evicting the oldest key requires scanning all entries in  $\mathcal{O}(N)$ !

Invariant

Order requires a list; instant lookup requires a map. Neither structure solves LRU alone.

# The LRU Solution: Map + Doubly Linked List

## Doubly Linked List Role

- **Recency Ordering:**  
**Head** = Most Recently Used (MRU).  
**Tail** = Least Recently Used (LRU).
- **Why Doubly Linked (not Singly)?**  
Having `node.prev` allows unlinking any node in  $\mathcal{O}(1)$  without scanning:  
`node.prev.next = node.next;`  
`node.next.prev = node.prev;`

## HashMap Role: Direct Pointer Index

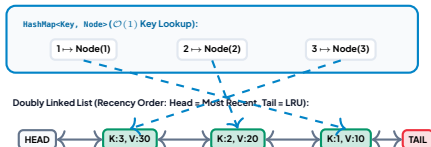
- **Type:** `HashMap<Integer, Node>`  
Maps each Key directly to its **memory node address** in the Doubly Linked List!
- **get(key):** Map lookup in  $\mathcal{O}(1) \implies$  detach and move node to Head in  $\mathcal{O}(1)$ .
- **put(key, val):** Update and move to Head. If full, evict `tail.prev` from list and map in  $\mathcal{O}(1)$ .

**Invariant**

The Map provides  $\mathcal{O}(1)$  random access; the Doubly Linked List provides  $\mathcal{O}(1)$  node repositioning.

# LRU Architecture: Sentinels & Pointer Invariants

## Composite System Diagram



## The Sentinel Node Trick

- Dummy HEAD and TAIL sentinels never store data.
- **Why Sentinels?** The list is **never empty**, completely eliminating all null checks during splicing or eviction!
- **Invariants:**
  - MRU is always `head.next`.
  - LRU is always `tail.prev`.

Invariant

Dummy sentinel nodes turn complex linked list edge cases into clean, universal  $\mathcal{O}(1)$  operations.

# LRU Cache: Node & Atomic Helper Operations

```
1 class LRUCache {
2     class Node {
3         int key, val;
4         Node prev, next;
5         Node(int k, int v) { this.key = k; this.val = v; }
6     }
7     private final int capacity;
8     private final Map<Integer, Node> map = new HashMap<>();
9     private final Node head = new Node(0, 0); // Dummy Head
10    private final Node tail = new Node(0, 0); // Dummy Tail
11
12    public LRUCache(int capacity) {
13        this.capacity = capacity;
14        head.next = tail;
15        tail.prev = head; // Initialize empty list
16    }
17    // Helpers continued on right...
18
```

```
1 // Atomic Helper 1: Unlink from anywhere
2 private void remove(Node node) {
3     node.prev.next = node.next;
4     node.next.prev = node.prev;
5 }
6
7 // Atomic Helper 2: Splice right after HEAD
8 private void insertAtHead(Node node) {
9     node.next = head.next;
10    node.prev = head;
11    head.next.prev = node;
12    head.next = node;
13 }
14
15 // Helper 3: Promote to MRU
16 private void moveToHead(Node node) {
17     remove(node);
18     insertAtHead(node);
19 }
20
```

**Invariant**

Factoring list surgery into 'remove' and 'insertAtHead' makes 'get' and 'put' trivial to implement.

# LRU Cache: Implementing get() and put()

```
1 public int get(int key) {  
2     Node node = map.get(key);  
3     if (node == null) return -1; // Cache miss  
4  
5     moveToHead(node); // Cache hit: promote to MRU  
6     return node.val;  
7 }  
8
```

## get(key) Invariant:

- $\mathcal{O}(1)$  map lookup finds node memory address.
- $\mathcal{O}(1)$  list promotion marks item recently accessed.

```
1 public void put(int key, int value) {  
2     Node node = map.get(key);  
3     if (node != null) { // Key exists: update & promote  
4         node.val = value;  
5         moveToHead(node);  
6     } else { // New key  
7         if (map.size() == capacity) { // Evict LRU  
8             Node lru = tail.prev;  
9             remove(lru);  
10            map.remove(lru.key);  
11        }  
12        Node newNode = new Node(key, value);  
13        insertAtHead(newNode);  
14        map.put(key, newNode);  
15    }  
16 }  
17
```

**Invariant**

Every operation in both 'get' and 'put' consists of 2–3 pointer updates and one hash operation: strictly  $\mathcal{O}(1)$ .

# LRU Cache: Step-by-Step Visual Dry Run

## State Transitions (Capacity = 2)

1. `put(1, 10)`: Map: {1}
2. `put(2, 20)`: Map: {1, 2}
3. `get(1) → 10`: [1] promoted to Head!
4. `put(3, 30)`: Cap full: [2] evicted!

## Trace Breakdown

- 1. `put(1, 10)`:  
Insert after Head  $\Rightarrow$  H  $\leftrightarrow$  [1]  $\leftrightarrow$  T.  
Map: {1}.
- 2. `put(2, 20)`:  
Insert after Head  $\Rightarrow$  H  $\leftrightarrow$  [2]  $\leftrightarrow$  [1]  $\leftrightarrow$  T. Map: {1, 2}.
- 3. `get(1) → 10`:  
Access key 1  $\Rightarrow$  unlink & splice after Head  
 $\Rightarrow$  H  $\leftrightarrow$  [1]  $\leftrightarrow$  [2]  $\leftrightarrow$  T.
- 4. `put(3, 30)`:  
Capacity full (2)! Evict `tail.prev([2])`.  
Insert [3] after Head  $\Rightarrow$  H  $\leftrightarrow$  [3]  $\leftrightarrow$  [1]  $\leftrightarrow$  T.

Invariant

The least recently accessed item naturally sinks to 'tail.prev', enabling instant eviction without scanning.

# Idea 4 Applications: Hashing Mastery Table

| Application                | Core Problem                                   | Role of Hashing / Invariant                                                              | Time                    | Space            |
|----------------------------|------------------------------------------------|------------------------------------------------------------------------------------------|-------------------------|------------------|
| Rabin-Karp Matching String | $\mathcal{O}(N)$ substring search in text      | Rolling polynomial hash (odometer slide: drop, shift, add)                               | $\mathcal{O}(N+M)$ avg  | $\mathcal{O}(1)$ |
| LRU Cache Architecture     | $\mathcal{O}(1)$ caching with recency eviction | Composite system: <code>HashMap&lt;Key, Node&gt;</code> + <code>Doubly LinkedList</code> | $\mathcal{O}(1)$ strict | $\mathcal{O}(C)$ |

**Algorithmic Hashing:** Rolling hashes convert sliding-window recomputations into  $\mathcal{O}(1)$  arithmetic updates.

**System / Composite Hashing:** Storing pointers inside a `HashMap` unlocks instant retrieval alongside dynamic doubly-linked list ordering.

**Invariant**

From algorithmic rolling hashes to composite system architectures, hashing provides constant-time superpowers.

# Session Summary

---

1. **The Hashing Pipeline:** Converting arbitrary keys to array slots via hash codes and modulo compression ( $\text{mod } M$ ), where collisions are resolved via Separate Chaining or Linear Probing.
2. **The Uniqueness Mindset (HashSet):** Leveraging  $\mathcal{O}(1)$  set membership to eliminate duplicates, detect state cycles, and identify sequence anchors ( $x - 1 \notin S$ ).
3. **The Frequency Mindset (HashMap):** Accumulating running counts via  $\text{getOrDefault}(k, 0) + 1$ , balancing anagram demands, and grouping strings via canonical signatures.
4. **The Complement Search Pattern:** Transforming quadratic pair searches (Two Sum) into single-pass  $\mathcal{O}(N)$  lookups via algebraic inversion ( $T - \text{nums}[i]$ ).
5. **Prefix Sum Subarrays:** Tracking running prefix sum frequencies with initial base case  $\{0 : 1\}$  to discover all  $K$ -sum subarrays in linear time via  $P_i = P_j - K$ .
6. **Algorithmic Rolling Hashes (Rabin-Karp):** Applying the odometer mental model (drop left, shift, add right) to achieve  $\mathcal{O}(N)$  average-time substring matching.
7. **Composite Architectures (LRU Cache):** Fusing a  $\text{HashMap}\langle \text{Key}, \text{Node} \rangle$  with a Doubly Linked List with dummy sentinels to guarantee strictly  $\mathcal{O}(1)$  get, put, and eviction.